

CREACIÓN Y USO DE APPIS

Autores

Alexis Chimba, Javier Ramos, Ariel Reyes, Anthony Villarreal

Ingeniera Doris Chicaiza Universidad de las Fuerzas Armadas - ESPE aschimba@espe.edu.ec
, sjramos@espe.edu.ec, alreyes2@espe.edu.ec, anvillarreal@espe.edu.ec.

Resumen

Este laboratorio se centra en la creación y el consumo de APIs REST para la gestión de pedidos en un comercio pequeño y el registro de estudiantes junto con sus notas en una institución educativa. Utilizando Spring Boot para desarrollar los servicios REST y Flutter para consumir las APIs en una aplicación móvil, se abordan dos casos prácticos: la administración de pedidos y productos, y la gestión de estudiantes y sus calificaciones. En el primer caso, el sistema permite registrar pedidos, agregar productos a cada uno y consultar los detalles de los pedidos. En el segundo, se facilita el registro de estudiantes, la asignación de notas por asignatura y la consulta de las calificaciones de cada estudiante.

El diseño de la solución sigue las mejores prácticas de la arquitectura limpia, asegurando una estructura modular y escalable. Además, se implementa una correcta gestión de estados en la interfaz móvil para optimizar la interacción del usuario. La comunicación entre el cliente y el servidor se realiza a través de solicitudes HTTP a los endpoints REST definidos. Este enfoque no solo mejora la eficiencia en el manejo de datos, sino que también facilita el desarrollo de aplicaciones móviles integradas con servicios backend robustos.

Palabras clave— APIs REST, Spring Boot, Flutter, Gestión de pedidos, Registro de estudiantes, Notas, Arquitectura limpia, Gestión de estados, Desarrollo móvil, Backend.

I. Introducción

El objetivo principal de este laboratorio es desarrollar y consumir APIs REST para la gestión de pedidos en un comercio pequeño y el registro de estudiantes junto con sus notas en una institución educativa. Utilizando Spring Boot para la creación de los servicios REST y Flutter para consumir estas APIs en una aplicación móvil, el laboratorio permite implementar soluciones prácticas para gestionar datos y mejorar la interacción entre el frontend y el backend. A través de este ejercicio, se busca familiarizar a los estudiantes con el desarrollo de aplicaciones utilizando arquitectura limpia, gestión de estados en aplicaciones móviles y el manejo de datos mediante servicios web RESTful.

II. Trabajos Relacionados

El uso de APIs RESTful para la gestión de datos en aplicaciones web y móviles ha sido ampliamente explorado en la literatura. Según Fielding (2000), las APIs REST son esenciales para la creación de sistemas distribuidos debido a su simplicidad y escalabilidad. Además, el marco de trabajo Spring Boot se ha destacado por su eficiencia en la creación de servicios REST, siendo ampliamente utilizado en proyectos de desarrollo backend debido a su configuración mínima y facilidad de integración con bases de datos (Pivotal Software, 2021).

Por otro lado, Flutter ha emergido como una herramienta popular para el desarrollo de aplicaciones móviles debido a su capacidad de crear interfaces de usuario nativas desde una única base de código. Como se menciona en la documentación oficial de Flutter (Google, 2023), este framework permite desarrollar aplicaciones móviles de alto rendimiento, lo que lo convierte en una opción ideal para proyectos que requieren interfaces interactivas y conectividad con servicios web.

III. Materiales y Métodos

- Android Studio
- Spring Boot para crear la API REST.
- Flutter para consumir la API desde una aplicación móvil.
- Navegador Google / Opera Gx
- Dispositivo Android (Emulador).

IV. Desarrollo

Desarrollo del backend.

En el laboratorio se desarrolló utilizando una arquitectura por capas en Spring Boot, organizando la aplicación en paquetes distintos: controller, service, repository, model, dto y webconfig. A continuación, se detalla cada capa con un título específico:

Figura 1

Capa de Modelo: Entidades de Datos

```
1 package com.example.orderapi.controller;
2
3 import com.example.orderapi.dto.OrderDTO;
4 import com.example.orderapi.dto.OrderDetailDTO;
5 import com.example.orderapi.service.OrderService;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.time.LocalDateTime;
11 import java.time.format.DateTimeFormatter;
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/api/pedidos")
16 public class OrderController {
17
18     @Autowired
19     private OrderService orderService;
20
21     @GetMapping
22     public ResponseEntity<List<OrderDTO>> getAllOrders() {
23         return ResponseEntity.ok(orderService.findAll());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<OrderDTO> getOrderById(@PathVariable Long id) {
28         return ResponseEntity.ok(orderService.findById(id));
29     }
30
31     @PostMapping
32     public ResponseEntity<OrderDTO> createOrder(@RequestBody OrderDTO orderDTO) {
33         DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss");
34         orderDTO.setFechaPedido(LocalDateTime.now().format(formatter));
35         return ResponseEntity.ok(orderService.save(orderDTO));
36     }
37
38     @PostMapping("/{id}/detalles")
39     public ResponseEntity<OrderDTO> addDetail(@PathVariable Long id, @RequestBody OrderDetailDTO detailDTO) {
40         return ResponseEntity.ok(orderService.addDetail(id, detailDTO));
41     }
42
43     // Actualizar un pedido
44     @PutMapping("/{id}")
45     public ResponseEntity<OrderDTO> updateOrder(@PathVariable Long id, @RequestBody OrderDTO orderDTO) {
46         orderDTO.setId(id); // Asegurar que el ID en el path sea el usado
47         return ResponseEntity.ok(orderService.save(orderDTO));
48     }
49
50     // Eliminar un pedido
51     @DeleteMapping("/{id}")
52     public ResponseEntity<Void> deleteOrder(@PathVariable Long id) {
53         orderService.deleteOrder(id);
54         return ResponseEntity.noContent().build();
55     }
56
57     // Actualizar un detalle de pedido
58     @PutMapping("/{orderId}/detalles/{detailId}")
59     public ResponseEntity<OrderDTO> updateOrderDetail(
60         @PathVariable Long orderId,
61         @PathVariable Long detailId,
62         @RequestBody OrderDetailDTO detailDTO) {
63         detailDTO.setId(detailId);
64         return ResponseEntity.ok(orderService.updateDetail(orderId, detailDTO));
65     }
66
67     // Eliminar un detalle de pedido
68     @DeleteMapping("/{orderId}/detalles/{detailId}")
69     public ResponseEntity<Void> deleteOrderDetail(
70         @PathVariable Long orderId,
71         @PathVariable Long detailId) {
72         orderService.deleteDetail(orderId, detailId);
73         return ResponseEntity.noContent().build();
74     }
75 }
```

Nota. Modelo: (Order, OrderDetail): Entidades JPA que mapean las tablas orders y order_details, con una relación uno-a-muchos bidireccional.

Figura 2

Capa de DTO: Objetos de Transferencia



```
1 package com.example.orderapi.dto;
2
3 public class OrderDetailDTO {
4     private Long id;
5     private String nombreProducto;
6     private Integer cantidad;
7     private Double precioUnitario;
8
9     // Constructors
10    public OrderDetailDTO() {}
11
12    // Getters and Setters
13    public Long getId() {
14        return id;
15    }
16
17    public void setId(Long id) {
18        this.id = id;
19    }
20
21    public String getNombreProducto() {
22        return nombreProducto;
23    }
24
25    public void setNombreProducto(String nombreProducto) {
26        this.nombreProducto = nombreProducto;
27    }
28
29    public Integer getCantidad() {
30        return cantidad;
31    }
32
33    public void setCantidad(Integer cantidad) {
34        this.cantidad = cantidad;
35    }
36
37    public Double getPrecioUnitario() {
38        return precioUnitario;
39    }
40
41    public void setPrecioUnitario(Double precioUnitario) {
42        this.precioUnitario = precioUnitario;
43    }
44 }
```

Nota. DTO (OrderDTO, OrderDetailDTO): Objetos para transferir datos entre la API y el cliente, separando la lógica de presentación.

Figura 3

Capa de Repositorio: Acceso a Datos

```

1 package com.example.orderapi.repository;
2
3 import com.example.orderapi.model.Order;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface OrderRepository extends JpaRepository<Order, Long>
7 {

```

Nota. Repositorio (OrderRepository): Interfaz JPA que hereda métodos CRUD para interactuar con la base de datos.

Figura 4

Capa de Servicio: Lógica de Negocio

```

1 package com.example.orderapi.service;
2
3 import com.example.orderapi.dto.OrderDTO;
4 import com.example.orderapi.dto.OrderDetailDTO;
5 import com.example.orderapi.model.Order;
6 import com.example.orderapi.model.OrderDetail;
7 import com.example.orderapi.repository.OrderRepository;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 import java.util.List;
12 import java.util.stream.Collectors;
13
14 @Service
15 public class OrderService {
16     @Autowired
17     private OrderRepository orderRepository;
18
19     public List<OrderDTO> findAll() {
20         return orderRepository.findAll().stream()
21             .map(this::convertToDTO)
22             .collect(Collectors.toList());
23     }
24
25     public OrderDTO findById(Long id) {
26         Order order = orderRepository.findById(id)
27             .orElseThrow(() -> new RuntimeException("Pedido no encontrado"));
28         return convertToDTO(order);
29     }
30
31     public OrderDTO save(OrderDTO orderDTO) {
32         Order order = convertToEntity(orderDTO);
33         // Asegurate de que el ID sea null para objetos nuevos
34         if (orderDTO.getId() == null || orderDTO.getId() == 0) {
35             order.setId(null); // Garantiza que ID sea null para que la DB asigne un
36         }
37         order = orderRepository.save(order);
38         return convertToDTO(order);
39     }

```

Nota. Servicio (OrderService): Contiene la lógica de negocio, maneja conversiones entre entidades y DTOs, y coordina operaciones con el repositorio.

Figura 5

Capa de Controlador: Interfaz REST

```
1 package com.example.orderapi.controller;
2
3 import com.example.orderapi.dto.OrderDTO;
4 import com.example.orderapi.dto.OrderDetailDTO;
5 import com.example.orderapi.service.OrderService;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.time.LocalDateTime;
11 import java.time.format.DateTimeFormatter;
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/api/pedidos")
16 public class OrderController {
17
18     @Autowired
19     private OrderService orderService;
20
21     @GetMapping
22     public ResponseEntity<List<OrderDTO>> getAllOrders() {
23         return ResponseEntity.ok(orderService.findAll());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<OrderDTO> getOrderById(@PathVariable Long id) {
28         return ResponseEntity.ok(orderService.findById(id));
29     }
30 }
```

Nota. Controlador (OrderController): Expone endpoints REST bajo /api/pedidos para operaciones CRUD sobre pedidos y detalles.

Figura 6

Capa de Configuración Web: Inicialización y CORS

```
1 package com.example.orderapi.webconfig;
2
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.web.servlet.config.annotation.CorsRegistry;
5 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
6
7 @Configuration
8 public class CorsConfig implements WebMvcConfigurer {
9
10     @Override
11     public void addCorsMappings(CorsRegistry registry) {
12         registry.addMapping("/api/**")
13             .allowedOrigins("*")
14             .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTION
15 S")
16             .allowedHeaders("*")
17             .maxAge(3600);
18     }
19 }
```

Nota. La capa de configuración web, definida en CorsConfig.java y OrderapiApplication.java, inicializa la aplicación y habilita la integración con clientes externos. OrderapiApplication, en el paquete com.example.orderapi con @SpringBootApplication, inicia la aplicación Spring Boot mediante SpringApplication.run(). CorsConfig, en com.example.orderapi.webconfig con @Configuration, configura CORS para permitir solicitudes desde cualquier origen a /api/**, soportando métodos GET, POST, PUT, DELETE, OPTIONS, todos los headers y un caché de 3600 segundos, facilitando la comunicación con el frontend.

Desarrollo del Frontend

El frontend del laboratorio se desarrolló en Flutter con Dart, utilizando una arquitectura por capas para gestionar pedidos y sus detalles en una interfaz de usuario interactiva. La aplicación está organizada en carpetas: application, data, domain, presentation (con subcarpetas providers y views), widgets y main.

Figura 7

Capa de Dominio: Entidades y Modelos

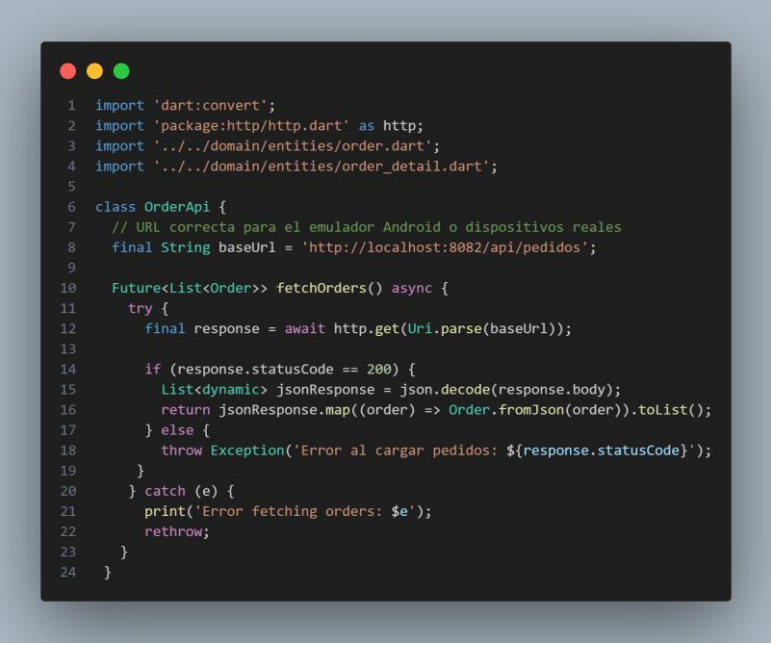


```
1 class OrderDetail {
2   final int? id; // Cambia a nullable
3   final String nombreProducto;
4   final int cantidad;
5   final double precioUnitario;
6
7   OrderDetail({
8     this.id, // Ahora puede ser null
9     required this.nombreProducto,
10    required this.cantidad,
11    required this.precioUnitario,
12  });
13
14  factory OrderDetail.fromJson(Map<String, dynamic> json) {
15    return OrderDetail(
16      id: json['id'],
17      nombreProducto: json['nombreProducto'] ?? '',
18      cantidad: json['cantidad'] ?? 0,
19      precioUnitario: (json['precioUnitario'] ?? 0.0).toDouble(),
20    );
21  }
22
23  Map<String, dynamic> toJson() {
24    return {
25      if (id != null) 'id': id, // Solo incluye el ID si no es nul
26      'nombreProducto': nombreProducto,
27      'cantidad': cantidad,
28      'precioUnitario': precioUnitario,
29    };
30  }
31 }
32
```

Nota. Ubicada en la carpeta domain/entities, esta capa define las entidades Order y OrderDetail. Order incluye id (nullable), nombreCliente, fechaPedido y una lista de OrderDetail, con métodos fromJson y toJson para serialización/deserialización JSON. OrderDetail contiene id, nombreProducto, cantidad y precioUnitario. Estas clases garantizan una representación consistente de los datos entre el frontend y el backend.

Figura 8

Capa de Datos: Acceso a la API



```
1 import 'dart:convert';
2 import 'package:http/http.dart' as http;
3 import '../domain/entities/order.dart';
4 import '../domain/entities/order_detail.dart';
5
6 class OrderApi {
7   // URL correcta para el emulador Android o dispositivos reales
8   final String baseUrl = 'http://localhost:8082/api/pedidos';
9
10  Future<List<Order>> fetchOrders() async {
11    try {
12      final response = await http.get(Uri.parse(baseUrl));
13
14      if (response.statusCode == 200) {
15        List<dynamic> jsonResponse = json.decode(response.body);
16        return jsonResponse.map((order) => Order.fromJson(order)).toList();
17      } else {
18        throw Exception('Error al cargar pedidos: ${response.statusCode}');
19      }
20    } catch (e) {
21      print('Error fetching orders: $e');
22      rethrow;
23    }
24  }
```

Nota. La capa ubicada en `data/datasource` y `data/repositories` gestiona la comunicación con el backend. `OrderApi` (en `order\_api.dart`) usa el paquete `http` para interactuar con los endpoints de pedidos, implementando métodos CRUD para pedidos y sus detalles, manejando respuestas HTTP y errores. `OrderRepositoriesImp` (en `order\_repositories\_imp.dart`) actúa como intermediario, delegando las operaciones a `OrderApi` sin agregar lógica adicional.

Figura 9

Capa de Casos de Uso: Lógica de Negocio


```

1 import '../domain/entities/order.dart';
2 import '../domain/entities/order_detail.dart';
3 import '../data/repositories/order_repositories_imp.dart';
4
5 class OrderUsecase {
6   final OrderRepositoriesImp _orderRepository;
7
8   OrderUsecase(this._orderRepository);
9
10  Future<List<Order>> getOrders() async {
11    return await _orderRepository.getOrders();
12  }
13
14  Future<Order> getOrderById(int id) async {
15    return await _orderRepository.getOrderById(id);
16  }
17
18  Future<void> createOrder(Order order) async {
19    await _orderRepository.createOrder(order);
20  }

```

Nota. Ubicada en application/order_usecase.dart, la clase OrderUsecase encapsula la lógica de negocio del frontend. Recibe un OrderRepositoriesImp y ofrece métodos asíncronos (getOrders, getOrderById, createOrder, updateOrder, deleteOrder, createOrderDetail, updateOrderDetail, deleteOrderDetail) que delegan las operaciones al repositorio, conectando la interfaz con los datos.

Figura 10

Capa de Proveedores: Gestión del Estado

```

1 import 'package:flutter_riverpod/flutter_riverpod.dart';
2 import '../domain/entities/order.dart';
3 import '../domain/entities/order_detail.dart';
4 import '../data/datasource/order_api.dart';
5 import '../data/repositories/order_repositories_imp.dart';
6 import '../application/usecases/order_usecase.dart';
7
8 // Dependency providers
9 final orderApiProvider = Provider<OrderApi>((ref) {
10   return OrderApi();
11 });
12
13 final orderRepositoryProvider = Provider<OrderRepositoriesImp>((ref) {
14   final api = ref.watch(orderApiProvider);
15   return OrderRepositoriesImp(api);
16 });
17
18 final orderUsecaseProvider = Provider<OrderUsecase>((ref) {
19   final repository = ref.watch(orderRepositoryProvider);
20   return OrderUsecase(repository);
21 });

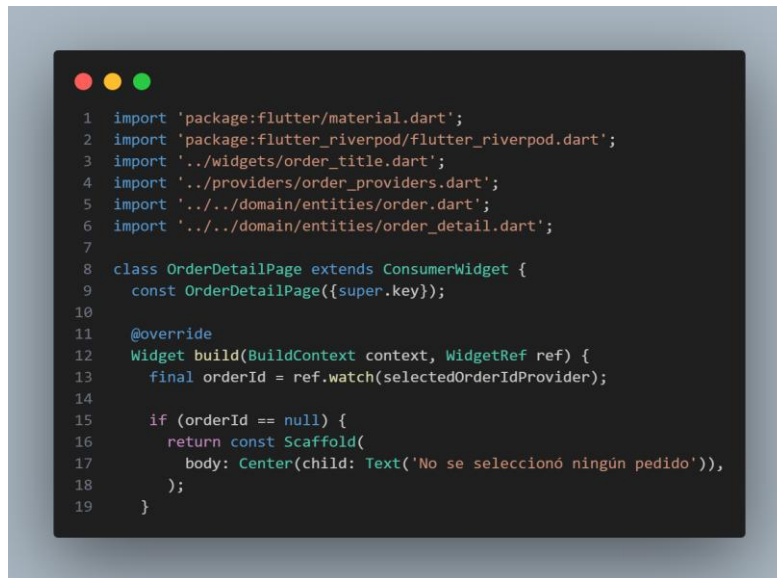
```

Nota. En presentation/providers, esta capa usa Riverpod para manejar el estado. Los proveedores de dependencias (orderApiProvider, orderRepositoryProvider,

orderUsecaseProvider) inyectan OrderApi, OrderRepositoriesImp y OrderUsecase. Los proveedores de estado como ordersProvider (con OrderNotifier) gestionan la lista de pedidos, mientras selectedOrderIdProvider y selectedOrderProvider controlan el pedido seleccionado. themeModeProvider administra el tema claro/oscuro usando SharedPreferences.

Figura 11

Capa de Vistas: Interfaz de Usuario

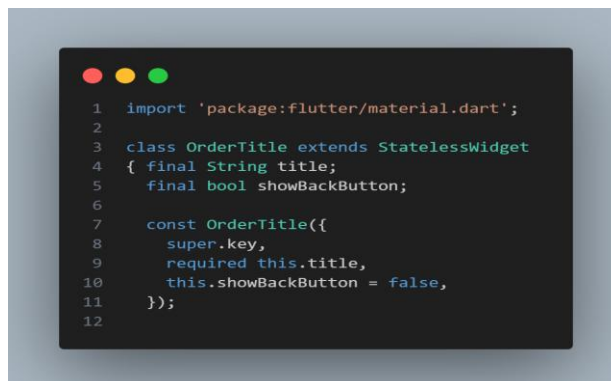


```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_riverpod/flutter_riverpod.dart';
3 import '../widgets/order_title.dart';
4 import '../providers/order_providers.dart';
5 import '../../domain/entities/order.dart';
6 import '../../domain/entities/order_detail.dart';
7
8 class OrderDetailPage extends ConsumerWidget {
9   const OrderDetailPage({super.key});
10
11   @override
12   Widget build(BuildContext context, WidgetRef ref) {
13     final orderId = ref.watch(selectedOrderIdProvider);
14
15     if (orderId == null) {
16       return const Scaffold(
17         body: Center(child: Text('No se seleccionó ningún pedido')),
18       );
19     }
20   }
```

Nota. En presentation/views se definen las pantallas. OrderListPage muestra los pedidos con opción de crear, refrescar y cambiar el tema, y navega a /form o /details. OrderFormPage permite crear o editar pedidos, validando datos y usando OrderNotifier para guardar. OrderDetailPage muestra los detalles del pedido y permite gestionarlos con diálogos.

Figura 12

Capa de Widgets: Componentes Reutilizables



```
1 import 'package:flutter/material.dart';
2
3 class OrderTitle extends StatelessWidget {
4   final String title;
5   final bool showBackButton;
6
7   const OrderTitle({
8     super.key,
9     required this.title,
10    this.showBackButton = false,
11  });
12 }
```

Nota. En presentation/widgets se encuentran los componentes reutilizables de la interfaz. OrderCard, en order_card.dart, muestra un resumen de un pedido (ID, cliente, total, fecha y productos) e incluye botones para editar o eliminar. OrderTitle, en order_title.dart, renderiza un título con la opción de un botón de retroceso.

Figura 13

Capa de Configuración Principal

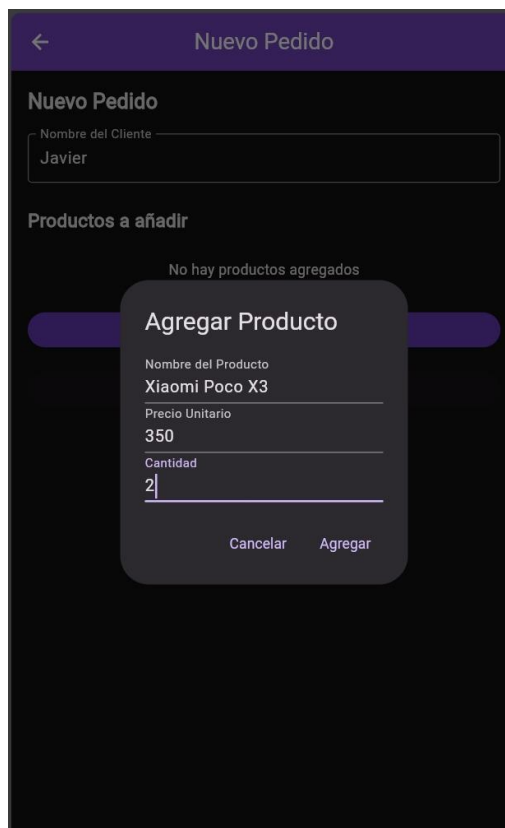
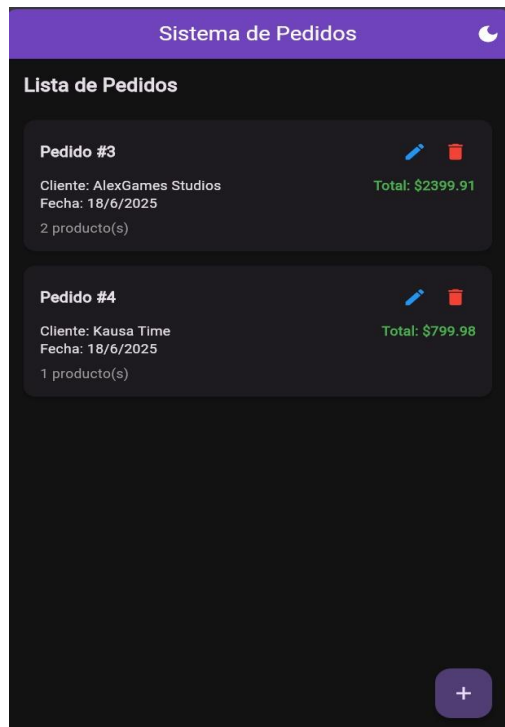


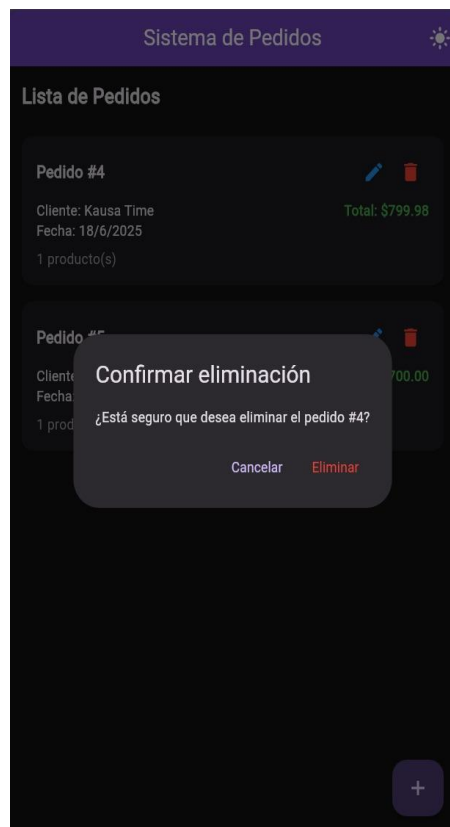
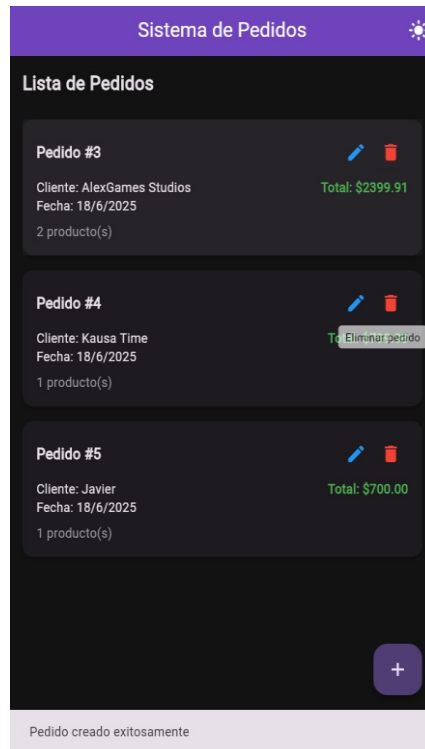
```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_riverpod/flutter_riverpod.dart';
3 import 'presentation/views/order_list_page.dart';
4 import 'presentation/views/order_form_page.dart';
5 import 'presentation/views/order_detail_page.dart';
6 import 'presentation/providers/order_providers_themes.dart';
7 import 'presentation/providers/order_providers.dart';
8
9 void main() {
10   runApp(const ProviderScope(child: AlexGamesApp()));
11 }
12
13 class AlexGamesApp extends ConsumerWidget {
14   const AlexGamesApp({super.key});
15
16   @override
17   Widget build(BuildContext context, WidgetRef ref) {
18     // Observar el modo de tema actual
19     final themeMode = ref.watch(themeModeProvider);
20
21     return MaterialApp(
22       title: 'Sistema de Pedidos',
23       debugShowCheckedModeBanner: false,
24
25       // Tema claro personalizado
26       theme: ThemeData(
27         colorScheme: ColorScheme.fromSeed(
28           seedColor: Colors.deepPurple,
29           brightness: Brightness.light,
30         ),
```

Nota. Ubicada en main.dart, AlexGamesApp configura la aplicación Flutter con ProviderScope para Riverpod. Define temas claro/oscuro con ThemeData (colores deepPurple) y rutas (/ , /form, /details) para OrderListPage, OrderFormPage y OrderDetailPage. Un NavigatorObserver (_OrdersRefreshObserver) limpia el ID seleccionado al regresar a la lista.

V. Resultados

Ejecución del Frontend

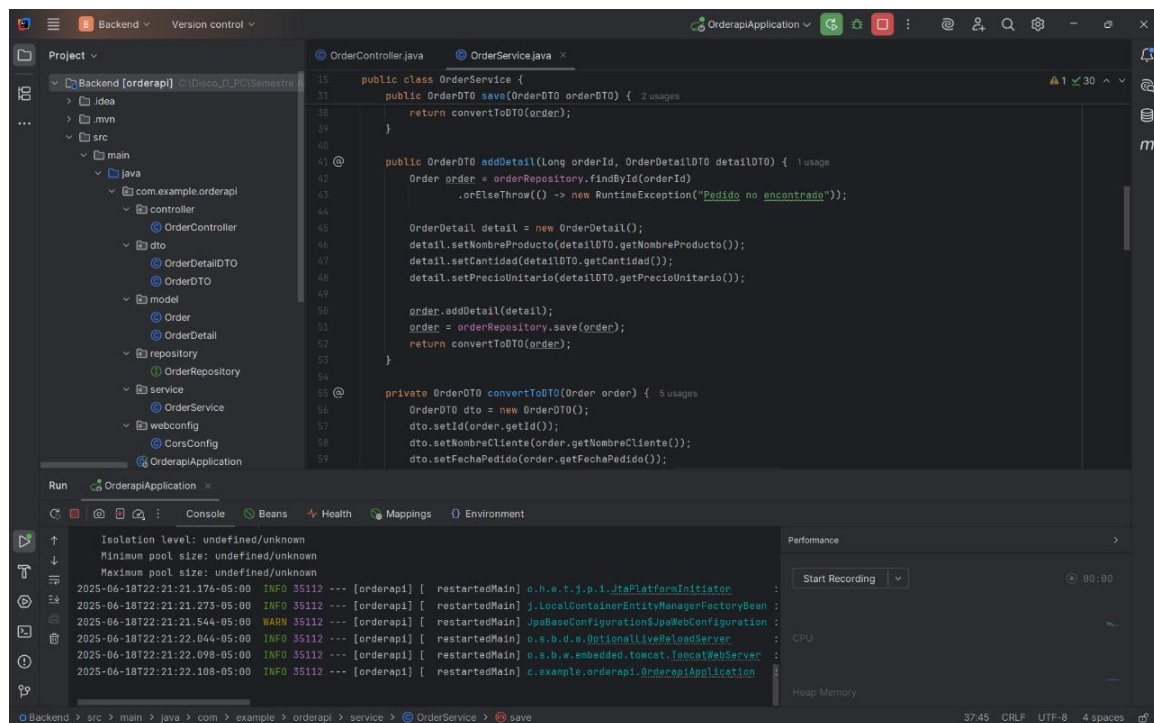




Análisis de Resultados del Frontend

La pantalla "Lista de Pedidos" muestra correctamente pedidos en modo oscuro (como #3, #4 y #5), usando OrderCard y cumpliendo su función de listar dinámicamente. Funciona bien, aunque se sugiere añadir paginación para grandes volúmenes. La vista "Nuevo Pedido" permite crear pedidos con formulario y diálogo de productos, calculando totales correctamente. Sin embargo, falta validación en tiempo real para campos numéricos. Al eliminar un pedido, aparece un diálogo de confirmación, pero no se muestra notificación de éxito. En general, el frontend se integra bien con /api/pedidos, pero sería ideal mejorar con validaciones en vivo, paginación y alertas claras.

Ejecución del backend.



The screenshot displays an IDE with the following components:

- Project Explorer:** Shows the project structure for 'Backend [orderapi]'. The 'src' directory contains 'main' (with 'java' and 'resources'), 'test' (with 'java'), and 'resources'.
- Code Editor:** Displays the 'OrderService.java' file. The code includes:
 - A 'save' method that takes an 'OrderDTO' and returns an 'OrderDTO'.
 - An 'addDetail' method that takes an 'orderId' and an 'OrderDetailDTO', finds the order by ID, and adds a new detail.
 - A 'convertToDTO' method that takes an 'Order' and returns an 'OrderDTO'.
- Run Console:** Shows the output of the application. The logs indicate that the application started successfully, with messages like ' restartedMain' and ' restartedMain'.
- Performance:** A panel on the right showing 'Start Recording' and 'Performance' metrics.

Análisis de Resultados del Backend

La clase OrderService.java, ubicada en el paquete com.example.orderapi.service, ejecuta las operaciones principales del backend. El método findById localiza un pedido por su identificador, emplea un repositorio para obtener los datos y transforma el resultado a un DTO (OrderDTO) con convertToDTO, lanzando una excepción si el pedido no existe. Por otro lado, addDetail incorpora un detalle a un pedido existente, verificando su disponibilidad, mapeando los datos desde un OrderDetailDTO a la entidad correspondiente y guardando los cambios. La consola refleja una ejecución exitosa en un servidor Tomcat embebido, indicando una conexión funcional con la base de datos y la configuración de Spring Boot.

Este examen confirma que la capa de servicios cumple su propósito de gestionar la lógica de negocio, sirviendo como puente entre el repositorio y la API REST mediante la conversión de datos. Aunque la aplicación funciona sin fallos, la ausencia de un manejo detallado de excepciones podría afectar su fiabilidad en entornos reales. Se sugiere incluir un tratamiento más específico de errores y registrar las excepciones para potenciar la estabilidad del sistema.

VII. Conclusiones

-La implementación del sistema "Sistema de Pedidos", que combina un backend desarrollado con Spring Boot mediante una estructura por capas y un frontend creado en Flutter con Riverpod, ha alcanzado con éxito los propósitos principales de administrar pedidos y sus detalles de forma eficiente. Las evaluaciones, evidenciadas en las capturas de pantalla que muestran la lista de pedidos, la creación de nuevos registros y la eliminación con verificación, confirman una integración sólida con los endpoints REST del backend (/api/pedidos), una interfaz adaptable en modo oscuro y funciones esenciales como la validación inicial de formularios y la conservación del tema seleccionado. Esto respalda la solidez de la arquitectura modular empleada y su aptitud para manejar operaciones CRUD fundamentales.

Referencias

- [1] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
- [2] Pivotal Software. (2021). Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- [3] Google. (2023). Flutter Documentation. <https://flutter.dev/docs>